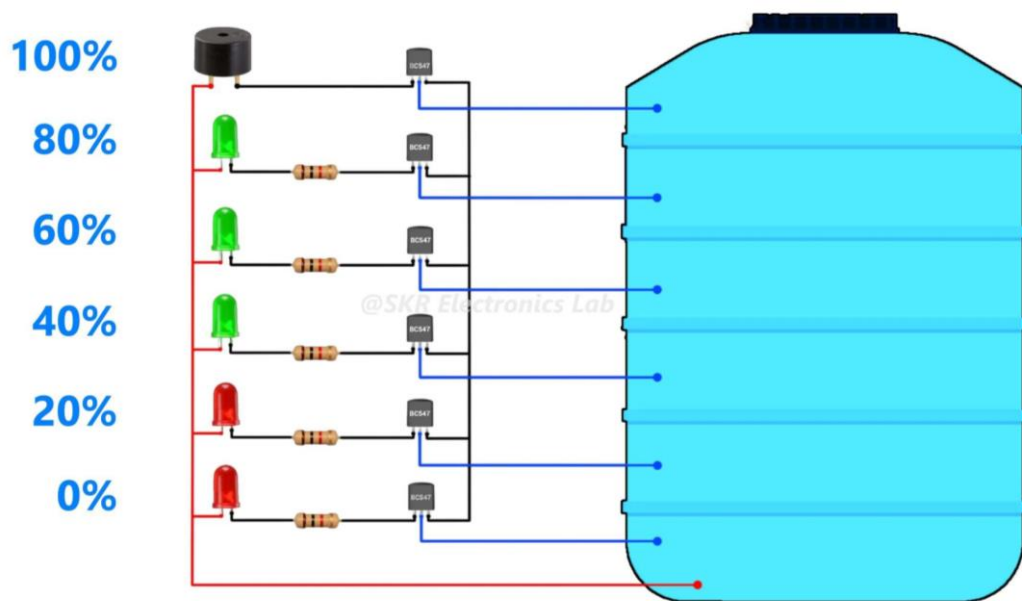




PROJECT REPORT

BASIC ENGINEERING (BAEEE – 101)

WATER LEVEL SENSOR



Submitted By:

Name – D SUBHAM KUMAR

Reg No. – 25BIT0275

Submitted To:

S VIVEKANANDAN

(Fall Semester, 2025-26)

CONTENTS

- ❖ OBJECTIVES
- ❖ METHODOLOGY
- ❖ COMPONENTS USED
- ❖ PROCEDURE
- ❖ RESULTS
- ❖ APPLICATIONS
- ❖ INFERENCE
- ❖ SOURCES USED

OBJECTIVES

1. To design and implement a water level sensor for monitoring the water level in a storage tank.
2. To indicate water levels (Low, Medium, High) using LEDs for better visualization.
3. To prevent water overflow and wastage by alerting when the tank is full using a buzzer.
4. To demonstrate the working principle of sensors and basic electronics using resistors, LEDs, and connecting wires.
5. To provide a simple, eco-friendly, and energy-efficient solution for household and industrial water management.

METHODOLOGY

1. Problem Identification

- Identified the need for a simple water level indicator system that shows the tank status (Low, Medium, High) and gives an alert when the tank is full.

2. System Design

- Planned the project using Arduino Uno as the microcontroller.

- Decided to use 3 LEDs to represent Low, Medium, and High water levels, and a buzzer to give an alarm when the tank is full.

3. Component Selection

- Chose Arduino Uno, LEDs, resistors, jumper wires, breadboard, and a buzzer for circuit implementation in Tinkercad.

4. Circuit Implementation

- Connected water level probes to Arduino input pins through simulated connections in Tinkercad.
- Assigned outputs from Arduino to control 3 LEDs and 1 buzzer.

5. Programming

- Wrote and uploaded an Arduino code in Tinkercad to control LEDs and buzzer according to water level signals.

6. Programmed logic:

- Low level → Green LED ON
- Medium level → Yellow LED ON
- High level → Red LED ON + Buzzer ON

7. Testing & Observation

- Simulated different water levels in Tinkercad.
- Verified that LEDs and buzzer worked correctly according to tank status.

COMPONENTS USED

1. Tinkercad Simulation Software

- Online platform used to design, implement, and test the water level indicator virtually.

2. Arduino Uno

- Acts as the main microcontroller.
- Receives signals from water level probes and controls LEDs and the buzzer according to programmed logic.

3. LEDs (Light Emitting Diodes) – 3 Nos.

- Used to indicate water levels.
- Green LED → Low level
- Yellow LED → Medium level
- Red LED → High level

4. Piezo (Buzzer)

- Provides an audible alarm when the water tank reaches the high level (full).

5. Resistors (220 Ω) – 3 Nos.

- Connected in series with LEDs to limit current and protect them from damage.

6. Jumper Wires

- Used to make electrical connections between different components.

7. Breadboard

- Used for prototyping the circuit without soldering.

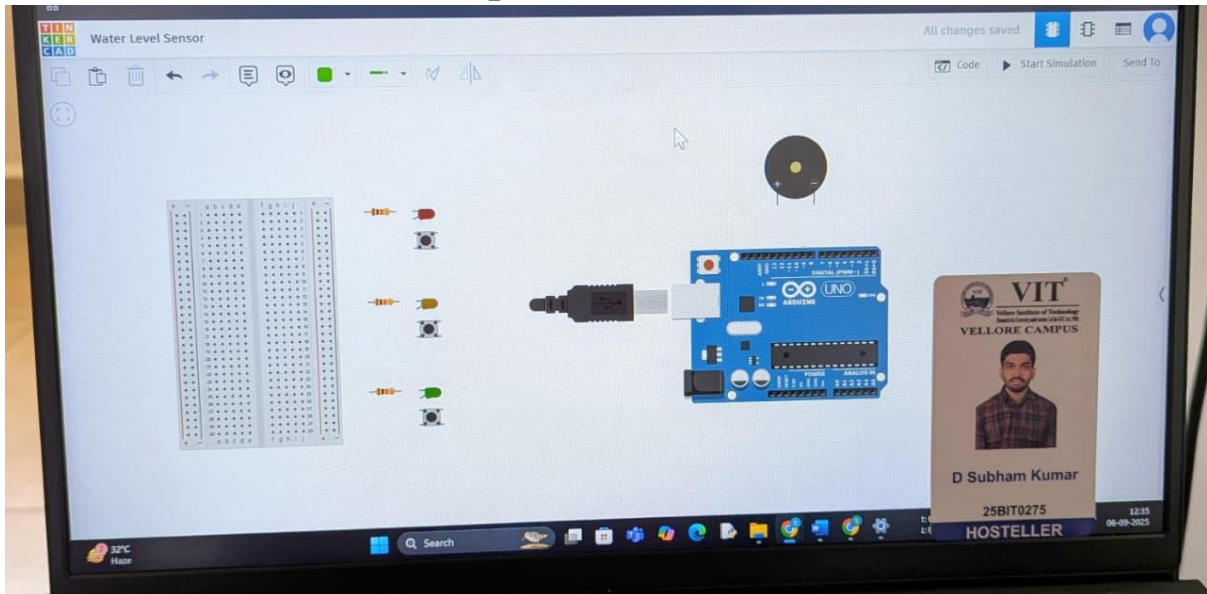
8. Pushbuttons – 3 Nos.

- Used as simulated water probes.
- Each button represents a probe at Low/Medium/High.

PROCEDURE

1. Prepare hardware in Tinkercad

- Place an Arduino Uno, a breadboard, 3 LEDs (green, yellow, red), $3 \times 220 \Omega$ resistors, 3 pushbuttons and 1 buzzer on the workspace as shown below.

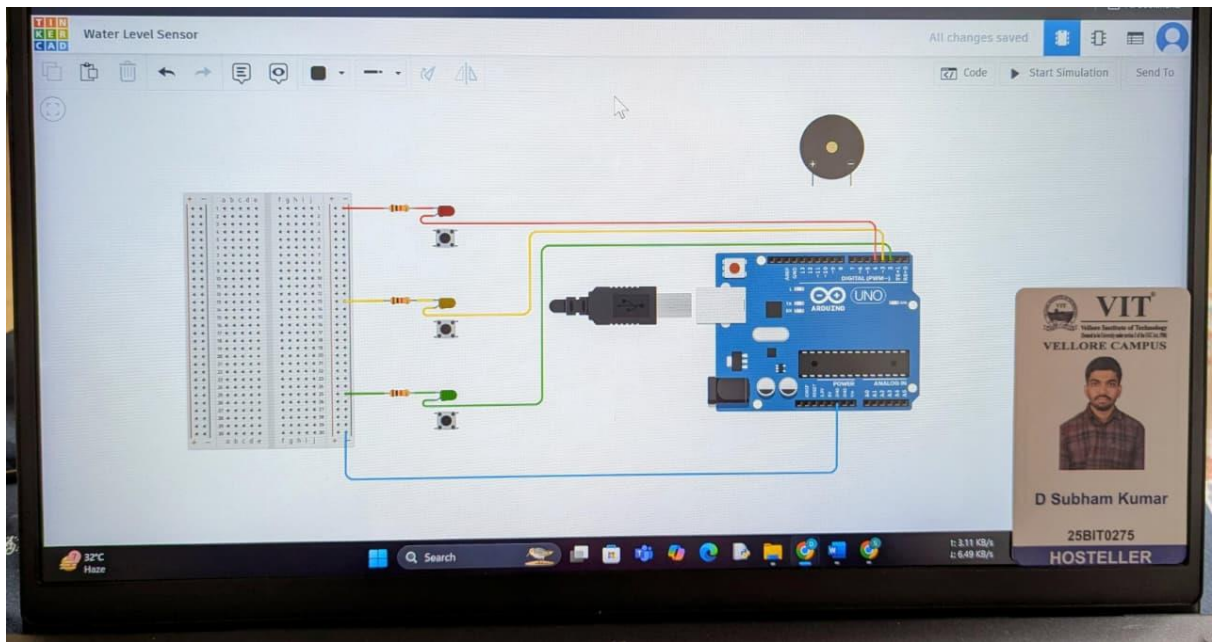


2. Connect the breadboard ground to Arduino GND

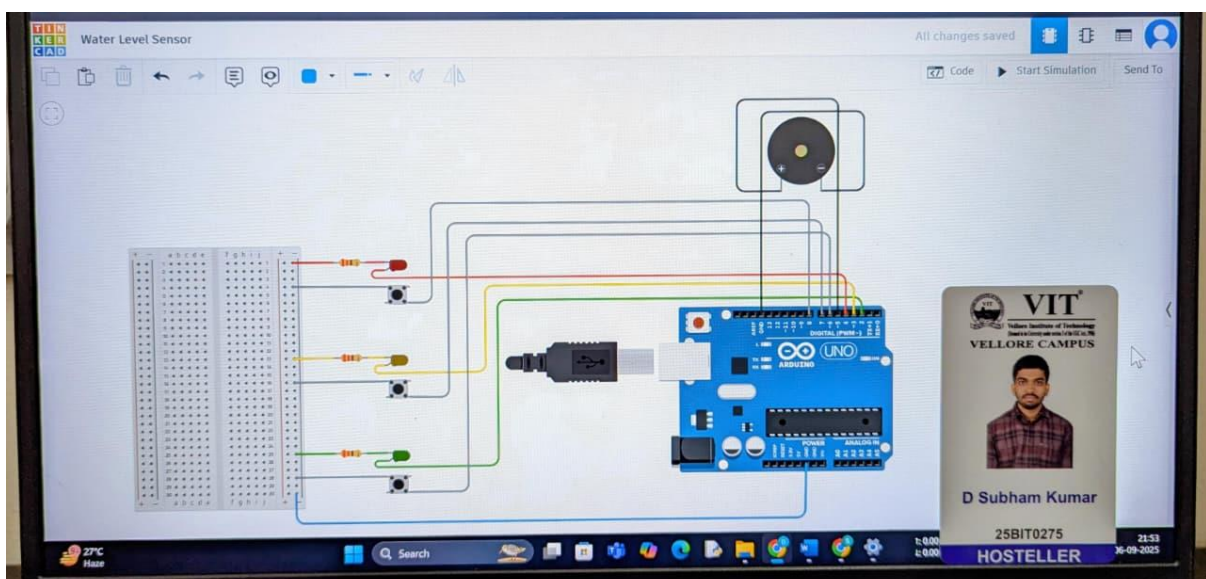
- Use a jumper from Arduino GND to the breadboard negative rail (–). This is the common ground for LEDs and buzzer.

3. Wire the LEDs

- Green LED (Low): Connect Arduino D2 pin to LED's anode (long leg) and LED's cathode (short leg) to breadboard GND rail via 220Ω resistor.
- Yellow LED (Med): Connect Arduino D3 pin to LED's anode and LED's cathode to GND rail via 220Ω resistor.
- Red LED (High): Connect Arduino D4 pin to LED's anode and LED's cathode to GND rail via 220Ω resistor. (Ensure LED orientation: long leg = anode to Arduino, short leg = cathode to GND.)



4. Wire the buzzer
 - Connect positive terminal of buzzer to Arduino D5 pin.
 - Connect negative terminal of Buzzer → breadboard GND rail.
5. Wire the pushbuttons (using internal pull-ups)
 - Rotate place button so you can use opposite diagonally pins. For each button:
 - Connect one terminal of button to Arduino digital input (Low = D6, Medium = D7, High = D8).
 - The other terminal to breadboard GND rail.



6. Paste & upload the Arduino code

- Open the code editor in Tinkercad for the Arduino and paste the C++ code below.

```
// Pins
const int greenLED = 2;
const int yellowLED = 3;
const int redLED   = 4;
const int buzzer    = 5;

const int lowSwitch  = 6; // Low probe (button)
const int medSwitch  = 7; // Medium probe (button)
const int highSwitch = 8; // High probe (button)

void setup() {
  // Outputs
  pinMode(greenLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(redLED, OUTPUT);
  pinMode(buzzer, OUTPUT);

  // Inputs with internal pull-ups (buttons wired to GND)
  pinMode(lowSwitch, INPUT_PULLUP);
  pinMode(medSwitch, INPUT_PULLUP);
  pinMode(highSwitch, INPUT_PULLUP);
}
```



```

    // Make sure everything starts OFF
    digitalWrite(greenLED, LOW);
    digitalWrite(yellowLED, LOW);
    digitalWrite(redLED, LOW);
    digitalWrite(buzzer, LOW);
}

void loop() {
    // Read switches (LOW = pressed)
    int lowState  = digitalRead(lowSwitch);
    int medState  = digitalRead(medSwitch);
    int highState = digitalRead(highSwitch);

    // Default: everything OFF
    digitalWrite(greenLED, LOW);
    digitalWrite(yellowLED, LOW);
    digitalWrite(redLED, LOW);
    digitalWrite(buzzer, LOW);

```

```

// Cascade logic: highest pressed level takes priority
if (highState == LOW) {           // High button pressed
    digitalWrite(greenLED, HIGH);
    digitalWrite(yellowLED, HIGH);
    digitalWrite(redLED, HIGH);
    digitalWrite(buzzer, HIGH);
}
else if (medState == LOW) {       // Medium button pressed
    digitalWrite(greenLED, HIGH);
    digitalWrite(yellowLED, HIGH);
}
else if (lowState == LOW) {       // Low button pressed
    digitalWrite(greenLED, HIGH);
}

delay(50); // small debounce / stability delay
}

```

7. Test the system

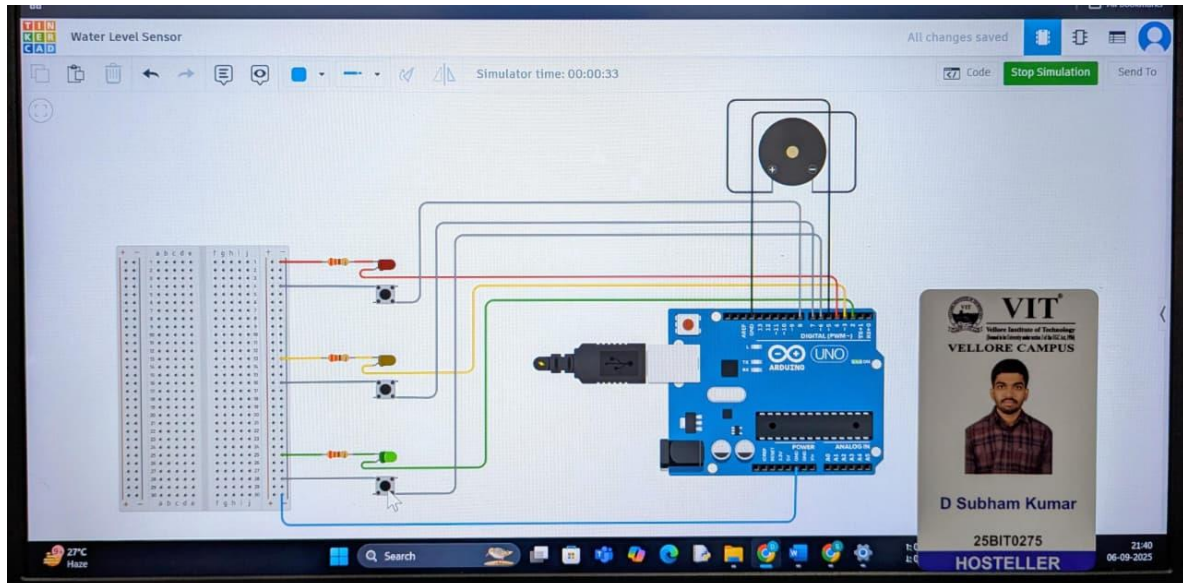
- Start the simulation: initially nothing should be ON.
- Press Low button.
- Press Medium button.
- Press High button.
- Pressing multiple buttons: the code prioritizes High > Medium > Low.

8. Record observations

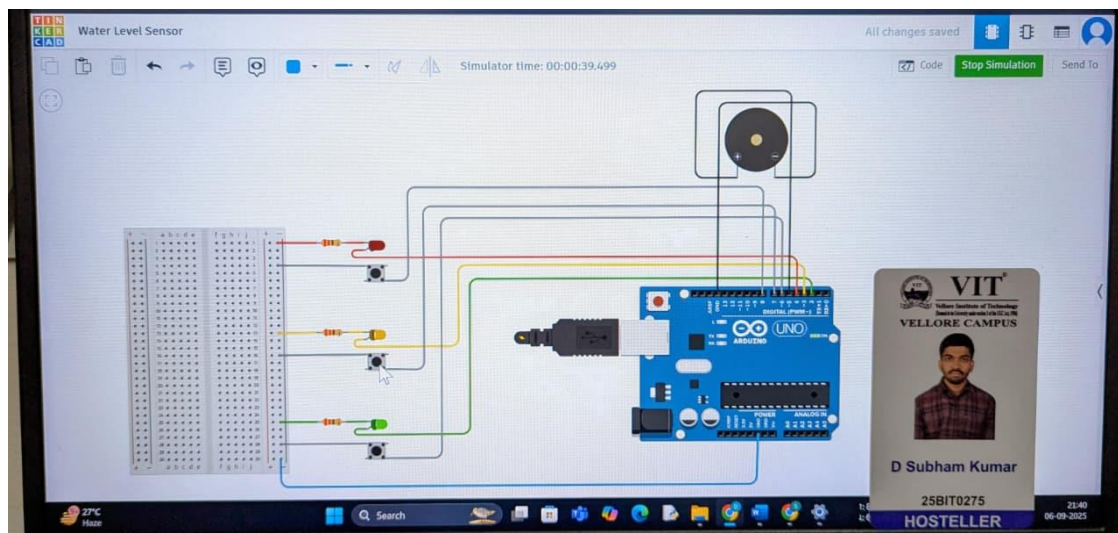
- Note the behavior when each button is pressed, and document any delays, buzzer tone, or stability issues.

RESULTS

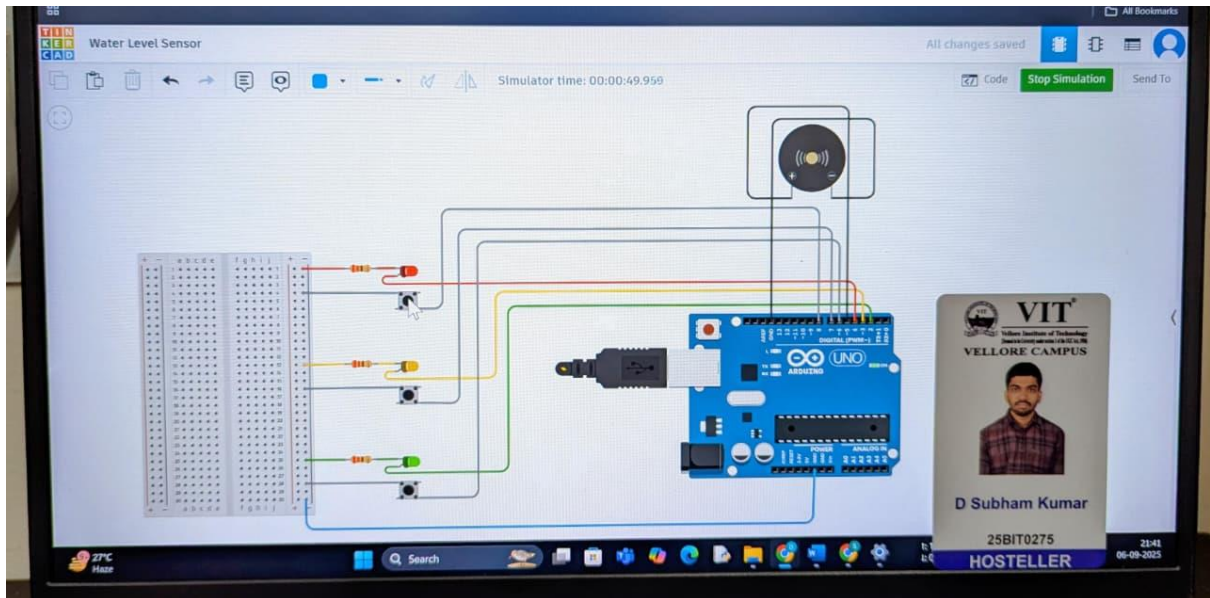
- ✓ The Green LED successfully glows when the Low level pushbutton is pressed, indicating that the tank has reached the minimum water level.



- ✓ The Yellow LED glows (along with the Green LED) when the Medium level pushbutton is pressed, showing the intermediate level of water.



- ✓ The Red LED glows (along with Green and Yellow LEDs) and the buzzer produces sound when the High level pushbutton is pressed, giving a visual as well as audible indication that the tank is full.



- ✓ The system consistently prioritizes the highest pressed level (High > Medium > Low) to avoid conflicting outputs.
- ✓ The circuit worked without false triggering because of the use of internal pull-up resistors, ensuring stable inputs.
- ✓ The simulation in Tinkercad verified the correctness of wiring and code, and the outputs matched the expected behavior.

Water Level	Input (Pushbutton)	Output (LEDs)	Buzzer status
Low	Low pressed	Green ON	OFF
Medium	Medium pressed	Green+Yellow ON	ON
High	High pressed	Green+Yellow+Red ON	ON

APPLICATIONS

1. Domestic water tanks – To monitor the water level and alert when the tank is full, preventing overflow.
2. Industrial reservoirs – To indicate and monitor liquid levels in storage tanks.
3. Agriculture – For irrigation systems to monitor water availability.
4. Overhead tanks in apartments – To provide floor-wise indication of water level.
5. Smart automation systems – Can be extended to automatically switch ON/OFF water pumps, preventing dry run or overflow.

INFERENCE

The water level indicator project using Arduino Uno, LEDs, pushbuttons (as probes), and a buzzer was successfully implemented in Tinkercad.

The simulation results confirmed that:

- ✓ The system correctly detected three levels (Low, Medium, High).
- ✓ Visual (LEDs) and audio (buzzer) outputs worked as expected.
- ✓ The use of Arduino simplified the logic implementation and ensured reliable performance.

Thus, the project demonstrates a low-cost, effective, and educational solution for water level monitoring, with potential for extension into real-world applications like automatic motor control and IoT-based smart water management.

SOURCES USED

1. **Tinkercad circuits** - Used for simulating the Arduino-based water level indicator project.
(<https://www.tinkercad.com/circuits>)
2. **ChatGPT by OpenAI** – Used for assistance in debugging, writing code, and preparing the report content.
3. **Arduino Documentation** – Used for understanding the internal pull-up resistor functionality and coding logic.
(<https://www.arduino.cc/reference/en/>)
4. **Online tutorials and forums** – Websites such as Arduino Project Hub and educational blogs for basic wiring and sensor interfacing.